Отчёт о производительности

Технический обзор производительности неофициального сервиса `rasp.ifmagic.ru`: адаптера личного расписания ИТМО, который отдаёт расписание подпиской ICS и read-only CalDAV.

1. Область и метод

Что измерялось. Два основных способа отдачи расписания и работа с базой данных:

- **подписка ICS** - отдача календарной ленты, которую периодически опрашивают календарные приложения;
- **read-only CalDAV** - получение отдельного объекта, перечисление коллекции (`PROPFIND`) и выборка объектов с данными (`REPORT calendar-query` с `calendar-data`);
- **запросы к базе данных** - планы выполнения горячих запросов до и после оптимизации.

Методика. Детерминированные данные, повторяемые серии запросов, сравнение «до и после» на одном и том же лабораторном стенде. Измерялись пропускная способность (запросы в секунду), задержки и перцентили, загрузка процессора, а также число прочитанных страниц базы данных в планах запросов. Отдельно сравнивалось размещение файлового кэша в оперативной памяти и на диске. Рабочий сервис не нагружался сверх ограниченного сценария чтения.

Границы. Это **внутренняя самооценка команды проекта (self-assessment)**, а не независимый аудит. Измерения сделаны на лабораторном стенде; на реальном сервере абсолютные значения будут иными, но структура результата сохраняется. Точные размеры тестовых данных, пороги и внутренние раскладки кэшей в отчёте намеренно не приводятся; сохранены только кратности ускорения и порядок пропускной способности.

2. Как отдаётся расписание

Расписание хранится в базе данных и отдаётся в двух видах: календарной лентой для подписки и коллекцией отдельных объектов для read-only CalDAV. Клиенты опрашивают сервис по своему расписанию, и в большинстве случаев у них ничего не меняется.

Исходная проблема была общей для обоих путей: **ответ пересобирался заново на каждый опрос**. Лента читалась из базы целиком даже тогда, когда клиенту достаточно было ответа «без изменений»; коллекция CalDAV на каждый запрос читала состояние событий и заново рендерила их в ICS.

Ключевые механизмы, которые устранили это:

- **файловый кэш отрендеренных лент.** Попадание отдаёт готовый результат через `http.ServeContent` (на Linux это использует `sendfile(2)`), промах рендерит ленту и атомарно сохраняет её в файл. На каждый запрос остаются только два лёгких обращения к базе - проверка профиля и версии расписания.

- **файловый кэш CalDAV-коллекции.** Один согласованный файл на профиль содержит метаданные событий и их тела; манифест и тела записываются одним `rename`, поэтому не могут разойтись. `PROPFIND` отвечает из манифеста без чтения тел, `GET` отдаёт одно тело, `REPORT` читает тела запрошенных событий.
- **условные запросы.** Поддержаны `ETag` / `304` (в том числе по `If-None-Match` и `If-Modified-Since`) и частичная отдача `Range`; ответ «без изменений» не читает тело.
- **согласованность кэша.** Инвалидация при отзыве, перевыпуске, административном удалении и изменении расписания; ограничение по сроку жизни и объёму с вытеснением по давности.

3. Результаты

3.1 Подписка ICS

Изначально отдача ленты упиралась в базу данных: узким местом было чтение всей ленты на каждый опрос, включая запросы, завершающиеся ответом «без изменений». Пропускная способность почти не росла с увеличением параллелизма.

Показатель	До	После
Пропускная способность ленты	порядка `10^3` запросов в секунду	порядка `10^4` запросов в секунду (примерно **10×**)
Зависимость от размера ленты	поток ограничен размером ленты и чтением из базы	поток **плоский**: большая и маленькая лента упираются в один потолок
Условный запрос (`304`)	не быстрее `200` : тело всё равно читалось для расчёта `ETag`	не дороже `200` ; тело не пересобирается
Частичная отдача (`Range`) и `HEAD`	не измерялось	сопоставимо с `200`
Узкое место	база данных	процессор приложения и HTTP-отдача

Попадание в кэш не обращается к тяжёлым данным и не рендерит ленту: остаются проверка профиля, проверка версии расписания и отдача файла. Именно это дало примерно десятикратный рост.

3.2 Read-only CalDAV

Для CalDAV добавлен кэш отрендеренной коллекции. Ускорение зависит от типа операции, потому что стоимость разных операций определяется по-разному.

Операция	До (кэш выключен)	После (кэш включён)	Ускорение
Получение одного объекта (`GET`)	порядка `10^3` запросов в секунду	порядка `10^4`	примерно **8.5×**
Перечисление коллекции (`PROPFIND Depth: 1`)	порядка `10^3`	порядка `10^3` - `10^4`	примерно **4×**
Выборка объектов с данными (`REPORT`)	порядка `10^2` - `10^3`	порядка `10^3`	примерно **1.6×**

Операция	До (кэш выключен)	После (кэш включён)	Ускорение
`GET` с `If-None-Match` (`304`)	-	сопоставимо с попаданием	-

`GET` и `PROPFIND` на попадании работают почти как отдача ленты: проверка профиля и версии плюс чтение файла. `PROPFIND` отвечает из метаданных и тела не читает. `REPORT` выигрывает меньше всех, потому что его основная стоимость - в большом ответе, который нужно собрать, XML-экранировать и передать клиенту, а не в обращении к базе.

3.3 Индексы базы данных

Отдельно проверялись планы выполнения горячих запросов. Нашли два места, где не хватало индекса на ссылающемся поле внешнего ключа: PostgreSQL индексирует первичный ключ родительской таблицы, но не сам столбец внешнего ключа. Из-за этого выполнялось полное сканирование таблиц.

Запрос (по назначению)	До	После
Эффективный интервал синхронизации по номеру студента	`Seq Scan`, порядка `10^0` мс, сотни страниц	`Bitmap Index Scan`, доли миллисекунды, единицы страниц
Удаление осиротевшего аккаунта	`Seq Scan` + проверка внешнего ключа, сотни страниц	`Index Only Scan` + проверка, единицы страниц
Список профилей с фильтром по номеру студента	`Seq Scan`, сотни страниц	`Index Only Scan`, единицы страниц
Каскадное удаление сессий при удалении профиля	проверка внешнего ключа полным сканом, сотни страниц	`Index Only Scan`, единицы страниц

Итог по затронутым запросам - ускорение примерно **20-60×** и сокращение числа прочитанных страниц с сотен до единиц. Остальные подозрительные запросы проверены и **опровергнуты**: резервирование задач воркера, проверка состояния синхронизации, поиск профиля по секрету, версия расписания и чтение состояния событий уже ходили по существующим индексам. Административные агрегаты и пагинация без фильтра - полные сканы по своей природе; индексы им не добавлены.

Цена решения: незначительный рост размера индексов и небольшое замедление вставки в микробенчмарке; для нагрузки сервиса это приемлемо.

3.4 Размещение кэша

При прогретом кэше скорость отдачи **не зависит от того, лежит кэш в оперативной памяти или на диске**: в обоих случаях данные читаются из системного кэша страниц, и разница в пределах погрешности. Размещение в памяти важно не для скорости, а для предсказуемого учёта ресурсов. При переполнении кэш деградирует мягко: запись пропускается с предупреждением, профиль обслуживается обычным путём, пользователь ошибки не видит.

4. Что это значит

- **Запас по ёмкости очень большой.** Пропускная способность многократно превышает расчётную нагрузку (порядок запаса - тысячи раз), поэтому горизонтальное масштабирование сейчас не требуется.
- **Узкое место сместилось.** Если раньше ограничением была база данных, то теперь на прогретом кэше - процессор приложения и отдача ответа по сети. Это более управляемая и дешёвая для масштабирования точка.
- **Ёмкость больше не зависит от размера данных.** На попадании большая и маленькая лента упираются в один потолок; тело перестало быть ограничением.
- **Кэш ускоряет и повторные обращения.** Первый запрос после перезапуска идёт обычным путём, последующие - из кэша.
- **Индексы дополняют кэш.** Они ускоряют фоновые и административные операции, а кэш снимает нагрузку с горячих пользовательских путей.
- **Лимиты - это политика, а не ограничение ёмкости.** Ограничения частоты нужны против злоупотреблений; запаса производительности хватает с большим избытком.

5. Что сделано по итогам

Область	Что было	Что сделано	Статус
Кэш лент	Каждый опрос читал всю ленту из базы, включая запросы с ответом «без изменений»	Файловый кэш отрендеренных лент; отдача готового файла через `http.ServeContent` (на Linux - `sendfile(2)`); `304` без чтения тела	Исправлено
Кэш CalDAV	`GET` / `PROPFIND` / `REPORT` на каждый запрос читали состояние событий и рендерили объекты	Файловый кэш коллекции: метаданные и тела в одном файле с атомарной записью; `PROPFIND` из манифеста, `GET` стримит одно тело	Исправлено
Условные запросы	Ответ «без изменений» был не дешевле полного	`ETag` / `304` (в том числе `If-Modified-Since`) и `Range`; `304` не читает тело	Исправлено
Индексы базы	Два внешних ключа без индекса давали полное сканирование	Добавлены аддитивные индексы на ссылающиеся поля	Исправлено
Согласованность кэша	Не было ограничений и инвалидации; конкурентные чтение и запись могли пересечься	Ограничение по сроку жизни и объёму с вытеснением; инвалидация при отзыве, перевыпуске, удалении и изменении; сериализация операций по профилю	Исправлено
Лимиты	Жёстко заданные значения по маршрутам	Настраиваемые лимиты по маршрутам и отдельный лимит по профилю	Исправлено

Область	Что было	Что сделано	Статус
Метрики	Не было метрик производительности	`GET /metrics` в формате Prometheus на отдельном внутреннем интерфейсе: HTTP, состояние воркера, метрики синхронизации	Исправлено
Метаданные ленты	Клиенты опрашивали ленту без подсказки о периодичности	Добавлены `REFRESH-INTERVAL` и `X-PUBLISHED-TTL`	Исправлено

6. Остаточные риски и границы

- **Кэш живёт в рамках одного экземпляра.** После перезапуска и на каждом новом экземпляре кэш прогревается заново; при нескольких экземплярах у каждого свой кэш. Для текущего масштаба это приемлемо.
- `REPORT` **ограничен размером ответа.** Это самая тяжёлая операция; её стоимость определяется объёмом передаваемых данных, а не базой. При необходимости возможен стриминг ответа или более узкий `time-range`.
- **Очень большие коллекции.** На профилях с тысячами событий растёт стоимость разбора метаданных на `GET`; это точка для будущей оптимизации, а не текущая проблема.
- **Холодный старт.** Первые запросы после перезапуска идут без кэша; это временно и не приводит к ошибке у клиента.
- **Абсолютные числа лабораторного стенда.** На реальном сервере абсолютные значения будут иными; структура результата (кэш снимает нагрузку с базы, узкое место - процессор и сеть) сохраняется. Реальный профиль сервера сверх ограниченного сценария чтения не нагружался.

7. Вердикт

За счёт файлового кэширования, поддержки условных запросов и точечных индексов сервис стал **примерно на порядок быстрее на подписке ICS** (порядок `10^4` запросов в секунду) и **в разы быстрее на CalDAV** (`GET` - примерно `8.5×`, `PROPFIND` - примерно `4×`, `REPORT` - примерно `1.6×`); горячие запросы к базе ускорились примерно `20-60×`. Узкое место сместилось с базы данных на процессор и сеть, а запас по ёмкости многократно превышает текущую и ожидаемую нагрузку.

Обзор носит характер **внутренней самооценки**. Открытых задач по производительности на момент публикации нет.

Внутренний обзор (self-assessment), не независимый аудит. Неофициальный сервис, не связан с Университетом ИТМО. Названия и товарные знаки принадлежат их владельцам.